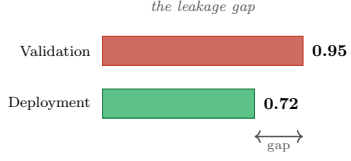
Applied Machine Learning

The Data Science Lifecycle

Data Leakage: What it is, how to spot it, how to avoid it



Learning goals

- Define data leakage and recognize its symptoms
- Identify the five major types: target, temporal, group, preprocessing, feature selection
- Apply the Golden Rule of preprocessing inside resampling
- Use a pre-deploy audit checklist before releasing a model

THE SILENT FAILURE MODE



Why this lecture exists.

▸ “Kapoor & Narayanan” 2023 surveyed ML applications across **17 scientific fields** and found data leakage in **294 papers**, often producing wildly overoptimistic conclusions and irreproducible findings.



The silent failure. Cross-validation reports the same overconfident performance the leaky model produces. The leak is *in the data*, not detectable from metrics alone.

WHAT IS DATA LEAKAGE?



Definition. Any information from outside the training distribution that influenced training, or information that would not be available at the moment of prediction.

Mechanism. Data that *would not be available* at prediction time leaks into the model during training.

Symptom. A large gap between cross-validation performance and held-out test performance, or implausibly good metrics.

Why it is silent. Cross-validation gives the same overconfident answer the model gives, because the leak is baked into the data, not into the procedure.

LEAKAGE TAXONOMY: FIVE TYPES



Type	Mechanism	Example	Fix
Target	Feature contains future info	`last_contact_reason = "cancellation_call"` used to predict churn	Audit features; remove or recompute
Temporal	Random CV on time-ordered data	Shuffled daily-revenue series across folds	Blocked / forward-chaining CV
Group	Same entity in train and test	Patient ID appears in both folds	Group-aware split
Preprocessing	Transformer fit on full data	Scaler computed on train + test combined	Pipeline inside CV loop
Feature selection	Selecting features on full data	Ranking features by target correlation on full data, then splitting	Nested CV / selection inside fold

Each of the five types gets a worked example next.

WORKED EXAMPLE: TARGET LEAKAGE



Scenario. Predicting customer churn from CRM records.

customer_id	last_contact_reason	age	churned (target)
A001	technical support	34	0
A002	**cancellation_call**	51	**1**
A003	billing question	28	0
A004	**cancellation_call**	42	**1**
A005	technical support	39	0

A naive model uses `last_contact_reason` and reports **0.98 AUC**. Looks fantastic.

The catch. At the moment we want to predict whether a customer *will* churn, the value `"cancellation_call"` does not exist for customers who have not yet churned. *The feature is the answer.*

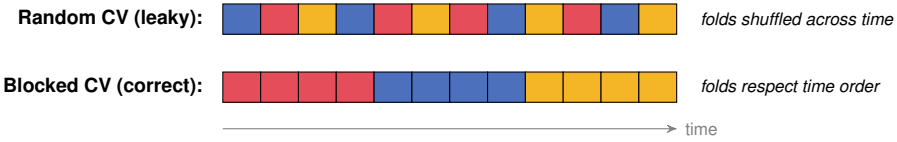
Detection rule. Audit each feature by asking:

Would this value exist at the moment of prediction?

WORKED EXAMPLE: TEMPORAL LEAKAGE



Scenario. Forecasting daily revenue from prior days' data.



With random k-fold CV, day 50 may end up in training and day 49 in the test fold. The model uses **the future to predict the past**, then fails in deployment where the future is unknown.

Fix: blocked CV or forward-chaining CV.

WORKED EXAMPLE: GROUP LEAKAGE



Scenario. Predicting hospital readmission. Each patient has multiple visits.

Naive split (leaky)

Train	P1	P1	P2	P3
Test	P1	P2	P3	

P1 appears in both!

Group-aware split (correct)

Train	P1	P1	P1	P2
Test	P3	P3		

each patient on one side only

Symptom. CV reports 0.92 accuracy; held-out cohort of new patients reports 0.71.

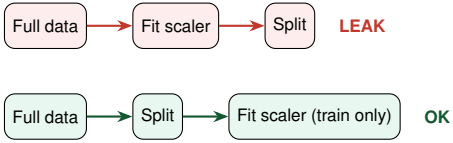
Fix. Split by group, not by row. In mlr3, assign a column the `group` role. Same logic applies to households, clinics, sessions, devices.

Leakage happens whenever CV folds violate the natural unit of generalisation.

PREPROCESSING LEAKAGE



The mechanism. Any transformer that *learns from data* (mean, variance, encoding mappings, PCA basis, k-means centroids) computed on the *full* dataset already reflects the test rows. The model has seen the test data's footprint before predicting.



Examples (technique names; not developed here): mean imputation, feature scaling, target encoding, dimensionality reduction, k-means binning.

You will encounter several of these in chapter 5 (feature preprocessing). The Golden Rule on the next slide applies to all of them.

FEATURE-SELECTION LEAKAGE: NULL SIMULATION



A worked example with *no true signal* – so we know exactly what an honest CV *should* report.

Simulation setup:

- y: exactly balanced binary target (50 of each class, then shuffled).
- X: $n = 100$ rows, $p = 10{,}000$ features, all i.i.d. standard normal.
- Every feature is *independent of* y by construction.
- Filter to top $k = 50$ features by univariate F-statistic.
- Fit a linear SVM; evaluate with 5-fold stratified CV; average over 50 seeds.

Expected outcome. An honest CV should report accuracy near **0.50** – there is no signal to learn.

Any clearly above-chance result must come from leakage, not real predictive structure.

FEATURE-SELECTION LEAKAGE: TWO WORKFLOWS



Run the same simulated data through two pipelines that differ only in *when* feature filtering happens.

Leaky workflow

1. Compute feature–target F-statistics on the *full dataset*.
2. Keep the top $k = 50$ features.
3. Run 5-fold CV on the already-filtered data.

Held-out labels influenced which features got selected.

Correct workflow

For each CV split:

1. Compute F-statistics on the *training fold only*.
2. Keep the top $k = 50$ features.
3. Fit the model on those features.
4. Evaluate on the held-out fold.

Held-out labels are never used during selection.

FEATURE-SELECTION LEAKAGE: SIMULATION RESULT



50 seeds of the null simulation:



(mean over 50 seeds; $n = 100$, $p = 10{,}000$, $k = 50$)

What just happened. On data with no real signal, the leaky pipeline reports near-perfect accuracy. Even the *worst* leaky run is 0.88; the worst proper run is 0.38.

Reading this. The correct procedure does *not* give "better performance" – it gives a more **honest estimate** of the pipeline's actual generalisation.

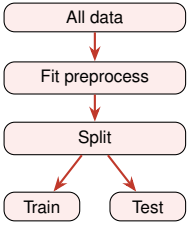
Mechanism = preprocessing. The selector is a transformer fit on data. **Fix:** re-rank inside each CV fold; wrappers that tune their own selection require nested resampling.

THE GOLDEN RULE



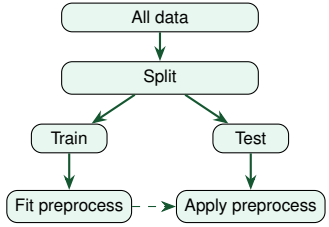
Every preprocessing step that *learns from data* must live **inside** the resampling loop.

LEAKY



Test data influenced the preprocess fit.

CORRECT



Preprocess fit on train only; applied to test.

- In **mlr3**: `po(...)` chains inside `as_learner(...)` enforce this automatically when used with `resample()` / `benchmark()`.
- In **scikit-learn**: wrapping preprocessing in `Pipeline` (optionally with `ColumnTransformer`) enforces this when used with `cross_validate` or `GridSearchCV`.

When in doubt: split first, fit later.

SPOTTING LEAKAGE: HEURISTICS AND AUDIT CHECKLIST



Detection heuristics

(post-hoc, on a finished model)

- **Big CV-test gap.** CV reports 0.95, holdout 0.80 $\Rightarrow$ suspect leakage.
- **Single feature too predictive.** Check its prediction-time availability.
- **Implausibly perfect performance.** Assume leakage until proven otherwise.
- **High-importance features.** Ask: “would I have this value at the moment of prediction?”

Pre-deploy audit checklist

(prospective, before shipping)

- ☐ Split before any preprocessing fit?
- ☐ CV folds respect group structure?
- ☐ CV folds respect time structure?
- ☐ Each feature available at prediction time?
- ☐ CV vs final-test gap reasonable?
- ☐ Feature selection inside CV?
- ☐ No duplicate rows across the split?